myNATEEJA

Welcome, GUDURI AMULYA

Logout

Evaluation Report :- GRANDTEST3-KMIT Final Score: 7/10 | Attempted: 5/5

Question 1 THEORY

Best Score: 8/10

What problem does multi-head attention solve?

Actual Answer

Multi-head attention allows the model to focus on different aspects of the input simultaneously. Each head learns different relationships, such as syntactic structure or semantic similarity. Instead of relying on a single attention distribution, multiple heads provide richer feature representations. This improves the model's ability to generalize and capture complex patterns. It essentially increases model expressiveness without a large computational cost.

Student's Answer

Multi-head attention is used in transformers to ensure each head is used to extract a unique feature, example: in a sentence, the semantics, verb adverb, syntax play a crucial role, therefore each head is used to extract the corresponding feature, in-order to get better outcomes.

GPT Evaluations

Evaluation 1 8/10

The student is demonstrating understanding of the same concepts addressed in the Actual Answer. They correctly explain that multi-head attention uses multiple heads to extract different features (semantics, syntax, etc.) from the input, which aligns with the idea of focusing on different aspects simultaneously and obtaining richer feature representations. They paraphrase the core purpose: each head learns unique relationships to improve outcomes. They did not explicitly mention the gains in generalization, capturing complex patterns, or the fact that this expressiveness comes at little extra computational cost, but they captured the main mechanism and goal accurately. Hence a score of 8/10 for good understanding with minor omissions.

Evaluation 2 8/10

The student is addressing the same concept as the Actual Answer—namely, that multi-head attention enables different heads to focus on different aspects of the input (semantics, syntax, etc.) to yield richer representations. They correctly explain in their own words that each head extracts a unique feature from the sentence, aligning with the

idea of learning different relationships. What they miss are some secondary points from the Actual Answer (the explicit mention of improved generalization, increased expressiveness without large computation cost, and replacement of a single attention distribution). Their response shows a solid grasp of the main purpose of multi-head attention but omits those additional benefits, so a score of 8 is appropriate.

Question 2 THEORY

Best Score: 3/10

Why is scaling applied in Scaled Dot-Product Attention, and what happens if we remove it?

Actual Answer

In attention, we compute the dot product between query (Q) and key (K). As the dimensionality (d_k) increases, these dot products can grow large in magnitude, pushing the softmax into regions with very small gradients. This leads to vanishing gradients and unstable training. To address this, the dot product is scaled by 1/sqrt(d_k), which keeps values in a reasonable range. Without scaling, training becomes slower and less stable, especially for large models. This simple normalization is critical for effective gradient flow.

Student's Answer

Scaling is applied at scaled dot-product attention because the output values after computing the dot product could be very high, few values might not be in a certain range. Therefore scaling ensures all the values have the mean of 0 and variance of 1, keeping all the values within a given range. If we do not apply scaling, few features might have high values and few may have low, to standardize we should scale the dot product attention

GPT Evaluations

Evaluation 1 3/10

The student is addressing the same concept—scaling the dot products in attention—but misunderstands the actual reason and mechanism. They correctly note that unscaled dot products can become large, but they incorrectly describe the scaling as standardizing to zero mean and unit variance rather than multiplying by $1/\sqrt{d_k}$. They also fail to mention the critical issue of softmax gradients vanishing and training instability without this scaling. Overall, their answer shows only a fragmentary and partly incorrect grasp of the Actual Answer's explanation.

Evaluation 2 3/10

The student is addressing the same concept—why we scale the dot products in attention—but their explanation shows significant misunderstandings compared to the Actual Answer. They incorrectly claim the purpose is to standardize to mean 0 and variance 1, as if applying z-score normalization, whereas the Actual Answer explains that scaling by $1/\sqrt{d_k}$ keeps large-magnitude dot products in a range where the softmax gradients remain large enough for stable, efficient training. The student also omits any mention of vanishing gradients or unstable training without scaling. Because they capture only the idea of keeping values in a bounded range but misunderstand the true rationale and fail to discuss gradient flow issues, this reflects weak understanding of the key mechanisms.

Question 3 THEORY

Best Score: 8/10

Why is positional encoding required in Transformers?

Actual Answer

Transformers process input tokens in parallel and do not inherently understand sequence order. Positional encoding adds information about the position of each token in the sequence. This allows the model to distinguish between sequences like “dog bites man” and “man bites dog.” These encodings can be sinusoidal or learned embeddings. Without positional encoding, the model would treat inputs as a bag of words, losing sequence context.

Student's Answer

In transformer, embeddings does not ensure the position of word in a given sentence, therefore to acknowledge the position of each word in a sentence, positional encodings were introduced that uses sin and cos and find its position, they are then added to the embedding. Positional encoding helps in knowing the position of a word in a sentence..

GPT Evaluations

Evaluation 1 7/10

The student is addressing the same concepts as the Actual Answer: that Transformers lack inherent sequence information in their parallel token processing, so positional encodings are needed to inject position data. They correctly note sinusoidal (sin/cos) encodings added to the token embeddings. However, they omit discussion of how without positional encoding the model would behave like a bag of words and fail to distinguish different token orders (e.g., “dog bites man” vs. “man bites dog”), and they do not mention learned embeddings as an alternative. Their explanation is accurate but

somewhat brief, showing good understanding of the core mechanism but missing some of the broader implications and details from the Actual Answer.

Evaluation 2 8/10

The student is addressing the same concept as the Actual Answer—Transformers lack inherent order information, so positional encodings (often sinusoids) are added to embeddings to inject token position. They correctly note that embeddings alone don't convey order and that sin/cos functions are used. They capture the core 'why' and 'how' but omit mention of the consequence (e.g. distinguishing 'dog bites man' vs. 'man bites dog') and the bag-of-words analogy. Thus they show good but not fully comprehensive understanding.

Question 4 THEORY

Best Score: 8/10

Why is dropout turned off during inference?

Actual Answer

Dropout is used during training to randomly deactivate neurons, preventing overfitting and encouraging robustness. During inference, all neurons are used to ensure consistent predictions. The weights are effectively scaled to account for dropout applied during training. Keeping dropout active during inference would introduce randomness in predictions. Therefore, it is disabled to ensure stable and deterministic outputs.

Student's Answer

Dropout is turned off during inference as the model is no longer in the training phase, where it could randomly remove certain neurons to allow other neurons to learn the features and help remove overfitting.. But in testing phase, the model is already trained and it found best weights and bias for prediction. Now in this phase if we turn on dropout, it may hamper the predictions and might not make correct predictions.

GPT Evaluations

Evaluation 1 8/10

The student is addressing the same concepts as the Actual Answer (why dropout is used in training and disabled in inference). They correctly explain that dropout randomly removes neurons to prevent overfitting during training and that enabling it at inference would hamper predictions. They also note that the model has fixed weights and biases at

test time. They miss the detail about scaling the weights to account for dropout during training and don't explicitly mention ensuring deterministic outputs, but they capture the core reasons. Hence, this is a good (7–8) level of understanding, meriting an 8.

Evaluation 2 **8/10**

The student is addressing the same concepts as the Actual Answer—namely that dropout randomly deactivates neurons during training to prevent overfitting and that it must be disabled at inference to avoid harming prediction quality. They correctly note that in the testing (inference) phase the model already has its trained weights and biases, and that applying dropout would introduce randomness and potentially produce incorrect predictions. They miss mentioning the detail that the weights are scaled to account for dropout during training and that disabling dropout ensures fully deterministic, consistent outputs. Overall, they demonstrate a solid grasp of the main ideas, with only minor omissions, so an 8/10 is appropriate.

Question 5 THEORY

Best Score: 8/10

Why does Random Forest reduce overfitting compared to a single decision tree?

Actual Answer

Random Forest reduces overfitting by combining multiple decision trees trained on different subsets of data (bagging). Each tree sees a slightly different dataset, which introduces diversity. It also randomly selects features at each split, preventing reliance on dominant features. The final prediction is an average (or majority vote), which smooths out noise. This reduces variance and improves generalization compared to a single tree.

Student's Answer

In random forest, multiple trees are used where a subset of data is used for training parallelly in each tree through bagging.. So, in this case, each tree would focus on only 1-2 features and make predictions accordingly then the max of all predictions will be considered as the output in case of classification and average of all outputs would be considered as the output in case of regression. Whereas in a single decision tree, the entire dataset is used in a single tree, where there might be many leaf nodes.. leading to increase in depth of tree and overfitting where the model memorizes the training dataset instead of learning the patterns. Therefore leading to overfitting

GPT Evaluations

GPT Evaluations

Evaluation 1 8/10

The student is addressing the same concepts as the Actual Answer (bagging with subsets of data, multiple trees, and combining predictions via vote or average) and thus is relevant. They correctly explain that each tree trains on a different subset of data (bagging) and that the ensemble's majority vote (classification) or average (regression) reduces overfitting compared to a single deep tree. They also contrast this with a single decision tree's tendency to overfit by memorizing the full dataset. However, they only partly capture the random feature-selection aspect (mentioning "1–2 features" but not clearly that features are chosen at random at each split to prevent reliance on dominant features) and do not explicitly discuss how averaging smooths out noise and reduces variance. These omissions are minor, so the explanation reflects good overall understanding with a small gap in describing feature randomness and noise reduction.

Evaluation 2 8/10

The student is addressing the same concepts as the Actual Answer: bagging with different data subsets, random feature selection, and combining tree outputs via majority vote or averaging. They correctly explain that each tree trains on a bootstrap sample and only a small random feature set, and that the ensemble prediction reduces overfitting compared to a single deep tree that memorizes the data. They miss an explicit mention of how averaging smooths out noise and reduces variance, but their answer shows a solid grasp of the main mechanisms by which Random Forest improves generalization. Hence, a score of 8 for good understanding with minor gaps.

© 2026 Teleparadigm Networks. All rights reserved.